



International University of Sarajevo

Faculty of Engineering and Natural Sciences

Database Management

CS306 Spring 2025

PROJECT 2 REPORT

Vector Database Implementation for AI Summarizer

Professor: dr. Ali Almisreb

Students: Berina Juković

Hena Šehović

Berin Žunić

Sarajevo, 01.06.2025.

Contents

1. Introduction.....	3
2. System Architecture.....	5
3. Critical Database Interactions	8
4. Integration with Database	12
5. User Interface and Example.....	15
6. Security and Strengths	19
7. Conclusion and Results.....	20
Table of Figures	21
References	21

1. Introduction

The goal of this project was to develop a web-based application that provides user the ability to generate summaries of the documents they give using advanced machine learning models. It uses natural language processing (NLP), user management for security, and an authentication system that is backed by SQLite. The application also has an option for users to ask question about the document they provided, and aims to provide accurate and multilingual summaries for everyone.

The project is an AI-powered PDF summarization system combines Streamlit (frontend), FastAPI (backend), and SQLite/ChromaDB (databases). Key components include:

- User authentication (registration/login)
- PDF processing (text extraction, chunking)
- AI summarization (topic-based Q&A, full-document summary)
- Vector database for semantic search

The project includes several key features designed to provide a secure, efficient, and user-friendly experience:

- **Vector Database Integration for Semantic Search and Retrieval**
The system is paired with a vector database that enables efficient storage and retrieval of text embeddings. From this, the app is able to perform sophisticated tasks such as semantic similarity search, which can be utilized to enhance the summarization process by identifying similar content, clustering similar documents, or providing contextual suggestions. The vector database is tasked with enabling the intelligent, scalable management of documents beyond the mere keyword match.
- **Secure user registration:**
The system includes a secure user management module that offers the ability to add new users and allow current users to log in securely. Passwords are never stored in plain text, but are hashed using industry-standard hashing algorithms so that sensitive information is secure even in the event of a data breach.
- **Multilingual Summarization Engine:**
It is powered by an mT5-based robust text summarization engine that is fine-tuned. This multi-language-supported state-of-the-art model enables users from diverse linguistic backgrounds to obtain concise and informative summaries of input text. The engine is fine-tuned for speed and accuracy, which makes it more usable over a wide range of content.

- Persistent Storage with SQLite:

All user information, including authentication information and relevant metadata, is stored in a lightweight yet powerful SQLite database. This is a compromise between strength and simplicity, enabling easy deployment without the overhead of full database servers yet still providing data integrity.

- Input Validation and Error Handling:

To provide a smooth and solid user experience, the project includes strict input validation at every stage. This repels typical errors, protects from malicious input, and notifies users with informative feedback, making the system stable and easy to use.

2. System Architecture

The AI Summarizer system consists of a dual-database design that is used for document retrieval and user administration, two functionally independent domains. This division between two databases ensures improved scalability, data integrity, and logical modularity between AI-based document summarizing engine and authentication processes.

a) SQLite Database (users.db)

The foundation for handling user identification and profile information is the SQLite database. It offers a serverless, lightweight, self-contained relational storage solution that is perfect for applications that need to be set up and deployed quickly.

```
cursor.execute('''
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        name TEXT NOT NULL,
        surname TEXT NOT NULL,
        telephone TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
''')
```

Figure 1: Schema of SQLite Database

Important attributes:

- Password Security: The SHA-256 hashing technique is used to securely store all user passwords, guaranteeing that plain-text passwords are never retained in the database.
- Validation of Input: All inputs are subject to stringent validation procedures. Email format validation, username restrictions (alphanumeric and underscores), and minimum password strength enforcement are all included in this.
- Constraints on Uniqueness: In order to prevent duplicate user accounts, the username and email fields are both need to be unique.
- Timestamps: For auditing and account management purposes, every user record is automatically timestamped at creation.

	id	username	email	password_hash	name	surname	telephone	created_at
1	1	ajdo	ajdo@gmail.com	8e80683b4300a49375beaaff8bdb48c55ea49a5d92fe870...	ajdin	ajdo	NULL	2025-06-01 16:19:54
2	2	tare	tarik@gmail.com	78a5587db28085259d5c4679f4c34eef025441ea4e4b75...	Tarik	Skaljic	+38762081616	2025-06-01 16:29:56
3	3	sabe	sabe@hotmail.com	f8bcb0797374486aabe364517a87cd7e808708aa1c609e5...	Saban	sabanovic	+3876231423	2025-06-01 16:26:31

Figure 2: Insight into SQLite Database

Figure 2 shows detailed view of the SQLite database schema used in the project. It shows the users table, which stores essential user information such as usernames, emails, hashed passwords, names, surnames, telephone numbers, and account creation timestamps.

This database is ideal for small to medium-sized online applications since it allows for effective login verification, account creation, and user lookup activities with no resource overhead.

b) ChromaDB Vector Database

Conceptually relevant text excerpts taken from uploaded PDF documents are stored and queried using ChromaDB. In AI-powered summarization workflows, it functions as a high-performance vector database designed for extensive similarity search and retrieval activities.

Storage structure works like following: every document is divided into smaller sections, usually consisting of paragraphs or groups of sentences, which are subsequently included into high-dimensional vectors. The following structure is used to store these chunks in a collection called pdf_documents:

```
processed_chunks.append({
    "id": chunk_id,
    "text": chunk.page_content,
    "metadata": metadata
})
```

Figure 3: Storage Structure

Each text chunk is converted into a numerical embedding using Google's newest `textembedding-gecko@001` model. These embeddings represent semantic meaning, allowing for advanced natural language understanding and relevance ranking.

Core operations include:

- Adding chunks: Upon upload of PDF, the document is parsed and broken up. Each chunk is embedded and stored with related metadata.
- Deleting chunks: Users can delete documents, and the corresponding chunks are deleted from the vector store.
- Semantic search: When the summarizer is queried by a user, their query is embedded in the same vector space. ChromaDB then retrieves the most similar text chunks according to cosine similarity or other distance metrics.
- Metadata exploitation: Metadata fields like `page_number` and `chunk_index` allow the system to keep contextual information, making it easy to have traceability and presentational sources of summary.

Among advantages is that this permits context-aware retrieval even when the query wording is divergent from that of the original content. It is extremely scalable as well to manage numerous users and big PDF document sets. And also delivers low-latency vector search performance that is optimized for machine learning use cases.

These databases work together to create a strong basis for the AI summarization platform, which combines the flexibility and strength of semantic vector search for document interpretation with the dependability of structured SQL storage for user data. Both developers and end users will have a smooth and safe experience thanks to this approach.

3. Critical Database Interactions

As explained in the previous part of the report application seamlessly coordinates operations between two key databases, previously mentioned as well, to ensure secure user management and efficient document processing. Below is a detailed explanation of the core interaction flows between the system components and databases.

a) User authentication

Users.db is a SQLite database that controls user authentication. Usernames, emails, hashed passwords, names, surnames, optional phone numbers, and timestamps of account creation are all stored in the relational database. The frontend, which was made with Streamlit, is where user registration starts. The information is sent to the backend over a FastAPI POST endpoint (/register/) whenever the user enters their information and submits the registration form. The create_user() method is run by the backend.

```
def create_user(user_data: Dict[str, Any]) -> Dict[str, Any]:
    """Create a new user in the database"""
    # Extract user data
    username = user_data.get('username', '').strip()
    email = user_data.get('email', '').strip().lower()
    password = user_data.get('password', '')
    name = user_data.get('name', '').strip()
    surname = user_data.get('surname', '').strip()
    telephone = user_data.get('telephone', '').strip() if user_data.get('telephone') else None
```

Figure 4: Creating a new user in the database

All input data is checked within this function, which is partially shown in figure 3. Usernames must only contain alphanumeric characters and underscores, and they must be between three and twenty characters long, the method that validates username is shown in figure 4. Passwords must be at least six characters long, and emails must follow the normal email format. Following successful validation, the user password is hashed using the SHA-256 technique to ensure that the plaintext password is never saved in the database.

```
def validate_username(username: str) -> bool:
    """Validate username format"""
    if len(username) < 3 or len(username) > 20:
        return False
    pattern = r'^[a-zA-Z0-9_]+$'
    return re.match(pattern, username) is not None
```

Figure 5: Validate username method

A parameterized SQL put statement, which guards against SQL injection threats, is used to put the hashed and signed data into the users table. Unique restrictions on the username and email columns provide protection against duplicate accounts.

Additionally, the login is secure. The credentials are collected by the frontend and sent to a login endpoint upon login. The backend then uses the `verify_user()` method to hash the password that was entered and compare it with the hash that is stored in the database, the methods part is shown in the following figure. After being matched, the user information is obtained and stored in Streamlit's `st.session_state`, which keeps the session open while the user interacts with the application. Strong validation, hashing, and session management work together to adequately secure user data.

```
def verify_user(username: str, password: str) -> Dict[str, Any]:  
    """Verify user credentials and return user data"""  
    if not username or not password:  
        raise ValueError("Username and password are required")
```

Figure 6: Method that verifies the user

b) PDF Processing

The application uses a second database, ChromaDB, which is a vector database for semantic search, to provide effective PDF summarization in addition to user management. When a user uploads a PDF file, libraries such as PyPDF2 are used by the system to scan the text content. LangChain's recursive character text splitter divides the entire corpus into smaller bits, or "chunks," because it is too big to handle efficiently in a single operation. To facilitate tracking and organizing, each chunk is additionally annotated with metadata, including the page number, chunk number, and source identity.

```
def chunk_text(text: str, chunk_size: int = 1000, chunk_overlap: int = 200) -> List[Dict[str, Any]]:  
    text_splitter = RecursiveCharacterTextSplitter(  
        chunk_size=chunk_size,  
        chunk_overlap=chunk_overlap,  
        length_function=len,  
        separators=["\n\n", "\n", " ", ""]  
    )  
    chunks = text_splitter.create_documents([text])
```

Figure 7: Usage of text splitter

Each chunk is kept in the pdf_documents ChromaDB collection. ChromaDB embeds text fragments into vector space to enable similarity searches. Google's embedding-001 model, which converts text to high-dimensional vectors of semantic meaning-preserved text, is used to accomplish this. Each such vector is added to the ChromaDB collection along with the metadata that goes with it and a unique name (such as "chunk_0"). By using semantic similarity instead of keyword matching, formal storage in this manner enables efficient and semantically relevant retrieval.

```
try:
    current_collection = chroma_client.get_or_create_collection(
        name=COLLECTION_NAME,
        embedding_function=gemini_ef
    )

    existing_ids = current_collection.get(ids=ids, include=[])[ 'ids' ]
    if existing_ids:
        current_collection.delete(ids=existing_ids)
        print(f"Deleted {len(existing_ids)} existing chunks with matching IDs.")

    current_collection.add(
        documents=documents,
        metadatas=metadatas,
        ids=ids
    )
    print(f"Successfully added {len(documents)} chunks to ChromaDB for '{file.filename}'.")
```

Figure 8: Document update logic

After it has been stored, users can type inquiries or questions in natural language to the system. They could type, "What is this paper about?" for instance. The same embedding approach is used to embed this as well as shown in figure 8, and the stored vectors are compared. ChromaDB uses a similarity score mechanism to return the chunks that are most semantically similar. The Gemini language model then uses these results, usually the top five, to translate and summarize the information in a natural language. The number of results used can be seen in the following figure in second last line of the code.

```

async def ask_pdf(query_data: QuestionQuery):
    user_question = query_data.question
    if not user_question.strip():
        raise HTTPException(status_code=400, detail="Please provide a question.")

    try:
        current_collection = chroma_client.get_or_create_collection(
            name=COLLECTION_NAME,
            embedding_function=gemini_ef
        )

        query_embedding_response = genai.embed_content(
            model="models/embedding-001",
            content=user_question,
            task_type="retrieval_query"
        )
        query_embedding = query_embedding_response['embedding']

        results = current_collection.query(
            query_embeddings=[query_embedding],
            n_results=5,
            include=['documents', 'distances', 'metadatas']
        )

```

Figure 9: The question-answering logic

The solution offers extensive document analysis with the guarantee that every step—from text extraction to vector embedding and query—is handled effectively thanks to such a well-organized workflow. In turn, ChromaDB and SQLite databases play different but equally important functions, providing safe user access and sophisticated, AI-powered summaries of intricate PDF documents.

4. Integration with Database

The main reason why the application works is the seamless integration of sophisticated AI models with structured datasets. With the help of this integration, the system may provide intelligent document analysis capabilities for submitted PDF documents, including subject modeling, summarization, and question-answering. The system heavily relies on ChromaDB for vector-based document storage and retrieval and Google's Gemini family of models for natural language processing.

The app is powered by two of Google's AI suite's most important models:

- The most recent gemini-1.5-flash: This is the most recent large language model used to generate excellent, contextually sensitive summaries of PDF content. It is perfect for real-time summarizing in a web environment because of its low latency and performance optimizations.
- models/embedding-001: It is an embedding model that converts raw text into high-dimensional numerical vectors with semantic context. The vectors are used for the purpose of similarity search in ChromaDB, enabling the system to pull out the most appropriate pieces of text for a specific query

```
# Configuration of Gemini
genai.configure(api_key=settings.google_api_key)
gemini_text_model = genai.GenerativeModel('models/gemini-1.5-flash-latest')

# Google Embedding Function for ChromaDB
class CustomGoogleEmbeddingFunction:
    def __init__(self, model_name: str = "models/embedding-001"):
        self.model_name = model_name
```

Figure 10: Configuration of Gemini and Embedding Function

A lot of AI-enhanced workflows are supported, each with a unique endpoint and unique interactions between the ChromaDB vector database and the AI models.

- Comprehensive Document Summarization at /summarize-pdf/
The backend pulls all document-related chunks from ChromaDB when a user views a PDF and chooses to get an overview. Passing the chunks to the gemini-1.5-flash-latest model gives an overview of the document's main ideas. Which is helpful for users who wish to get a sense of a full file without reading through all the details.

```

@app.post("/summarize-pdf/")
async def summarize_pdf():
    try:
        current_collection = chroma_client.get_or_create_collection(
            name=COLLECTION_NAME,
            embedding_function=gemini_ef
        )

        results = current_collection.get(ids=current_collection.get()['ids'], include=['documents'])

        if not results['documents']:
            raise HTTPException(status_code=404, detail="No PDF content found in the database to summarize. Please upload a PDF first.")

        full_document_text = "\n\n".join(results['documents'])
        summary = await summarize_text_with_gemini(full_document_text)

        return JSONResponse(content={
            "message": "PDF summary generated successfully.",
            "summary": summary,
            "summarized_chunks_count": len(results['documents'])
        })
    except HTTPException as he:
        raise he
    except Exception as e:
        print(f"Error in /summarize-pdf/ endpoint: {e}")
        raise HTTPException(status_code=500, detail=f"An error occurred during summarization: {e}")

```

Figure 11: Summarize-pdf

- **Topic-Based Summarization /summarize-topic/**
The user can enter a topic that interests them for more detailed information requirements. After creating an embedding of the topic query using the models/embedding-001 model, the system does a semantic search in ChromaDB to identify the most pertinent text passages. The Gemini model then receives this data and generates a focused summary based just on the designated topic. Using this option to browse documents by themes or keywords can be really of help.

```

@app.post("/summarize-topic/")
async def summarize_topic(query: Dict[str, str]):
    user_query = query.get("topic_query")
    if not user_query:
        raise HTTPException(status_code=400, detail="Please provide a 'topic_query' in the request body.")

    try:
        current_collection = chroma_client.get_or_create_collection(
            name=COLLECTION_NAME,
            embedding_function=gemini_ef
        )

        results = current_collection.query(
            query_texts=[user_query],
            n_results=5,
            include=['documents', 'distances', 'metadatas']
        )

        retrieved_chunks = results['documents'][0] if results['documents'] else []
        retrieved_metadatas = results['metadatas'][0] if results['metadatas'] else []

        if not retrieved_chunks:
            return JSONResponse(content={
                "message": "No relevant content found in the PDF for your topic. Please try a different query.",
                "summary": "Could not find relevant information to summarize for the given topic.",
                "retrieved_chunks_info": []
            })
    
```

Figure 12: Summarize-topic

- /ask-pdf/ Retrieval-Augmented Generation (RAG) for Question Answering
Users can give natural language queries on the submitted PDF thanks to this sophisticated approach. The embedding model is injected into the query, and ChromaDB is queried for semantically related text blocks. The Gemini model then uses a RAG (Retrieval-Augmented Generation) technique to provide the found sections as context. Based on the content of the paper, Gemini adjusts the context and generates an accurate, human-crafted response. Static PDFs can be effectively converted into interactive knowledge bases with this capability.

```
@app.post("/ask-pdf/")
async def ask_pdf(query_data: QuestionQuery):
    user_question = query_data.question
    if not user_question.strip():
        raise HTTPException(status_code=400, detail="Please provide a question.")

    try:
        current_collection = chroma_client.get_or_create_collection(
            name=COLLECTION_NAME,
            embedding_function=gemini_ef
        )

        query_embedding_response = genai.embed_content(
            model="models/embedding-001",
            content=user_question,
            task_type="retrieval_query"
        )
        query_embedding = query_embedding_response['embedding']

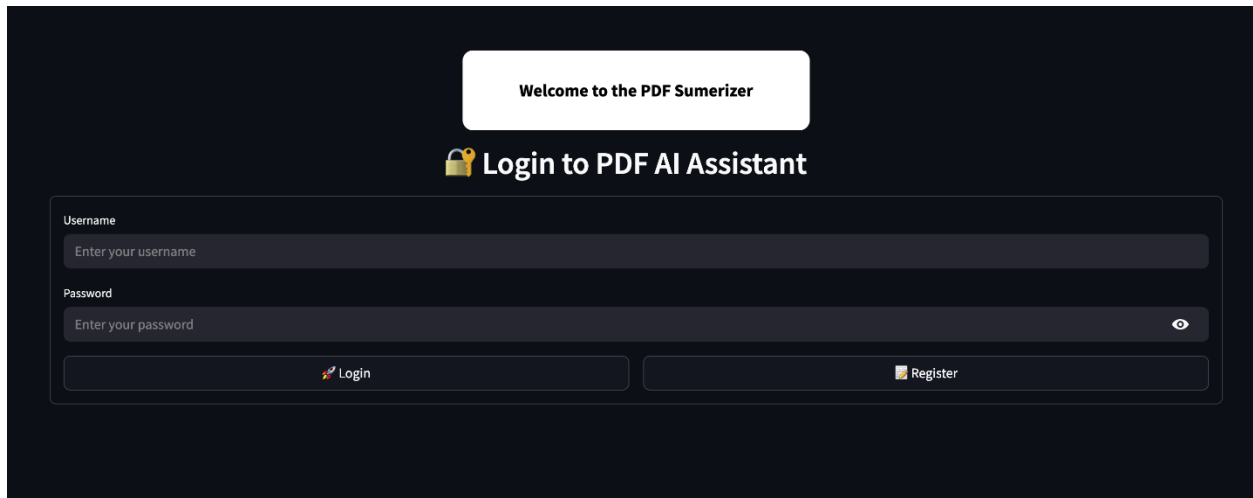
        results = current_collection.query(
            query_embeddings=[query_embedding],
            n_results=5,
            include=['documents', 'distances', 'metadatas']
        )
```

Figure 13: Ask-pdf

In addition to providing summaries, the system provides intelligent, interactive insights from complex documents by combining artificial intelligence abilities with relational and schema-based vector databases. The capability of merging AI with efficient data management for next-generation document processing applications can be seen by this endpoint-based, modular design, in which each endpoint represents a unique intent-based operation.

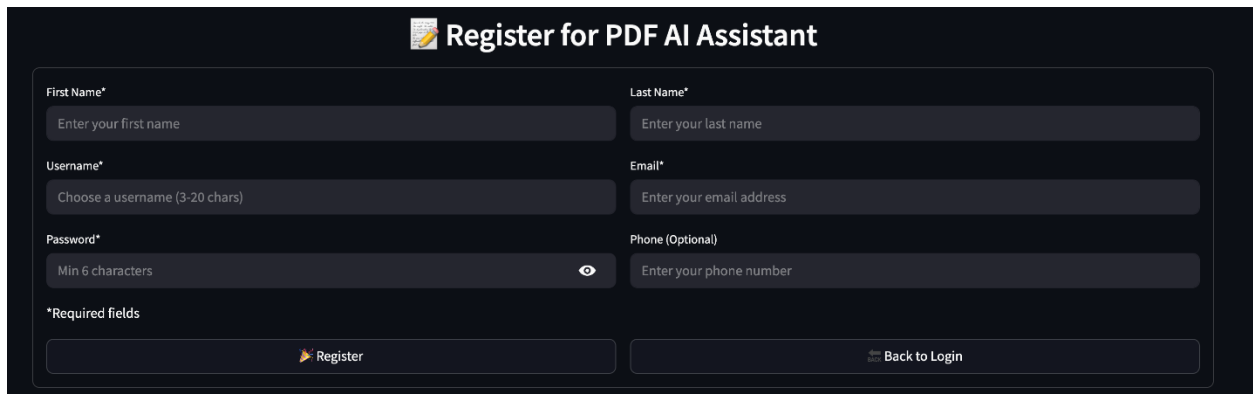
5. User Interface and Example

In this part of the project report we will show how graphic user interface looks like and an example of how the application works in the following figures. In figure 14, we can see the applications welcome page where user can either login, if they already have an account, or register to create an account. In the next figure, the registration form is showed, it can be seen that users personal information is needed for it.



The image shows the 'Welcome to the PDF Sumerizer' page. At the top, a white box contains the text 'Welcome to the PDF Sumerizer'. Below this, a yellow padlock icon is followed by the text 'Login to PDF AI Assistant'. The main form area has a dark background and contains two input fields: 'Username' with the placeholder 'Enter your username' and 'Password' with the placeholder 'Enter your password' and an eye icon for toggling visibility. At the bottom of the form, there are two buttons: 'Login' with a yellow arrow icon and 'Register' with a yellow document icon.

Figure 14: Welcome Page of the Application



The image shows the 'Register for PDF AI Assistant' page. At the top, a yellow document icon is followed by the text 'Register for PDF AI Assistant'. The form is divided into two columns. The left column contains 'First Name*' (placeholder: 'Enter your first name'), 'Username*' (placeholder: 'Choose a username (3-20 chars)'), and 'Password*' (placeholder: 'Min 6 characters' with an eye icon). The right column contains 'Last Name*' (placeholder: 'Enter your last name'), 'Email*' (placeholder: 'Enter your email address'), and 'Phone (Optional)' (placeholder: 'Enter your phone number'). At the bottom, there are two buttons: 'Register' with a yellow arrow icon and 'Back to Login' with a yellow document icon. A note '*Required fields' is located above the buttons.

Figure 15: Registration for AI Assistant

The next figures show the main page of the application, figure 16 showing on the left-hand side the users username and email address, and below it instructions. On top of the page we can see an options bar, where there are options for for uploading the PDF document, full summary, topic analysis, ask question and database.

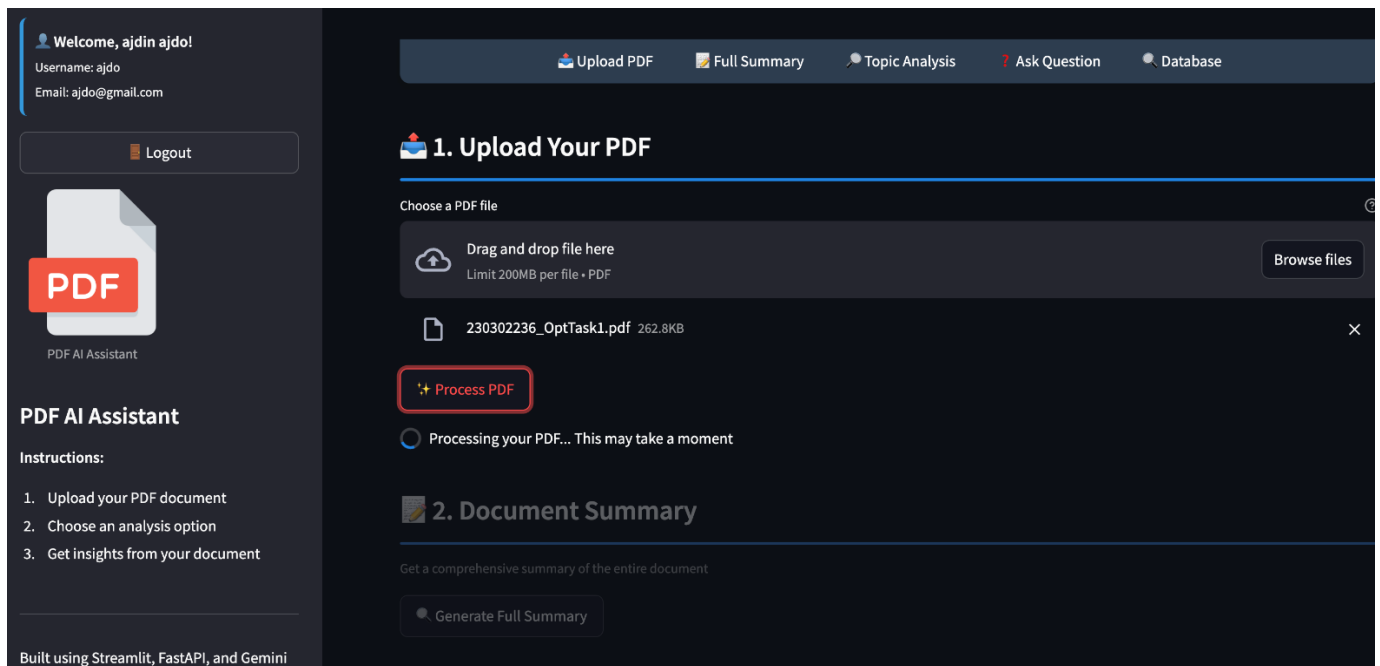


Figure 16: Upload PDF interface while processing

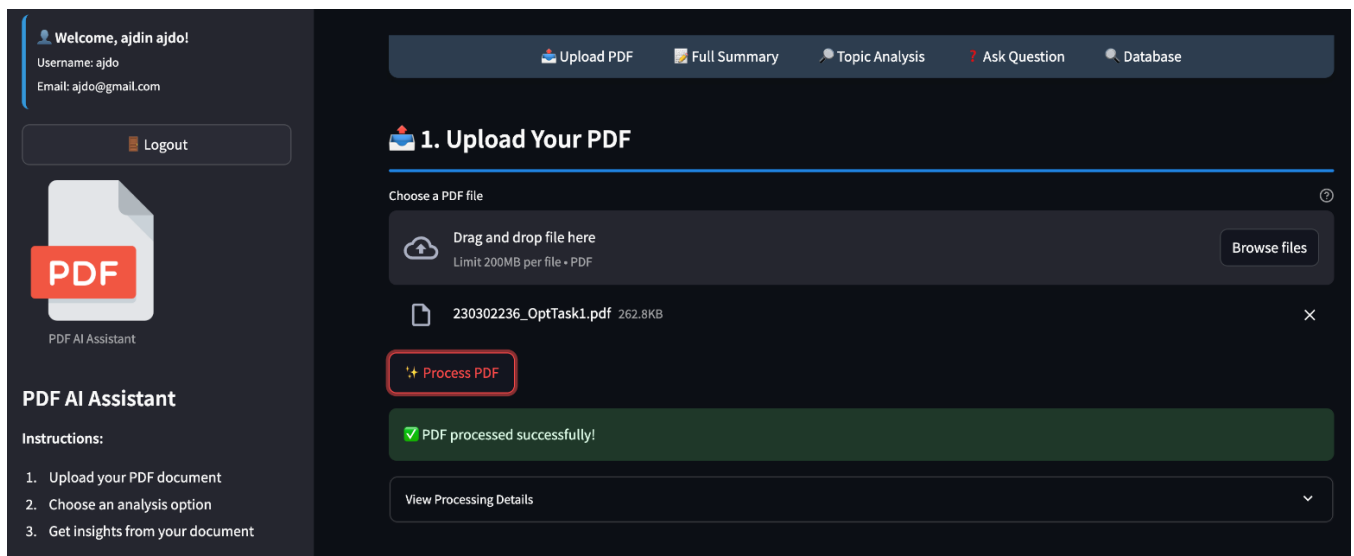


Figure 17: Interface after successfully processing the PDF

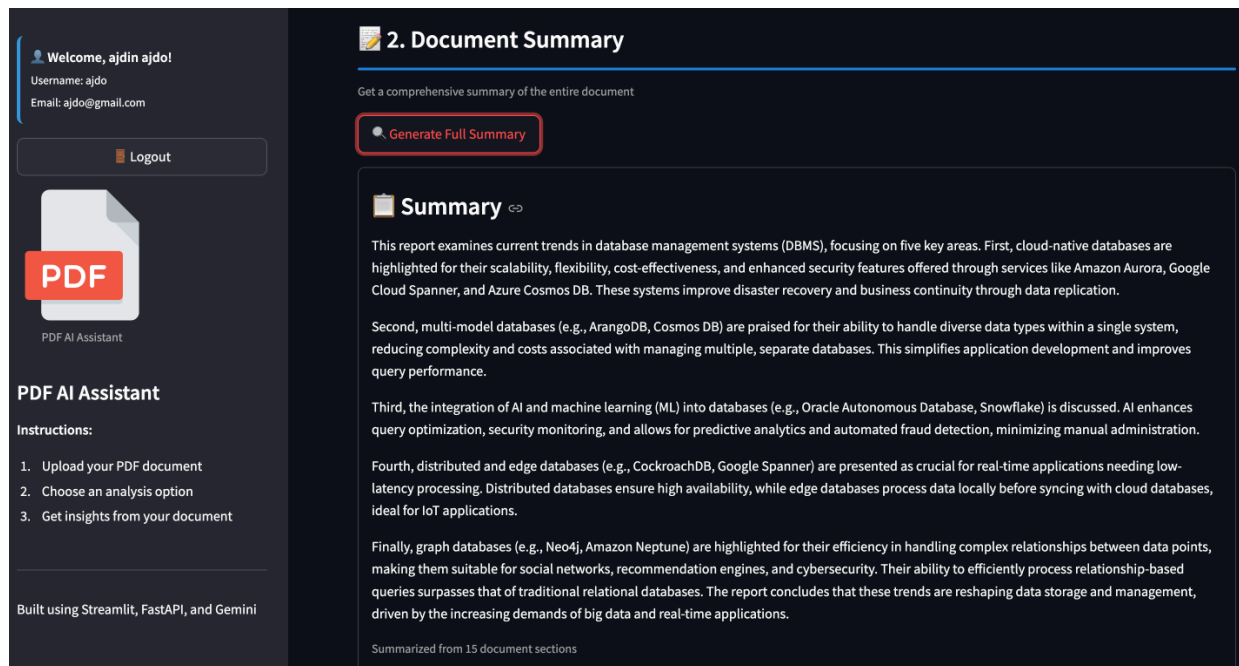


Figure 18: Document Summary

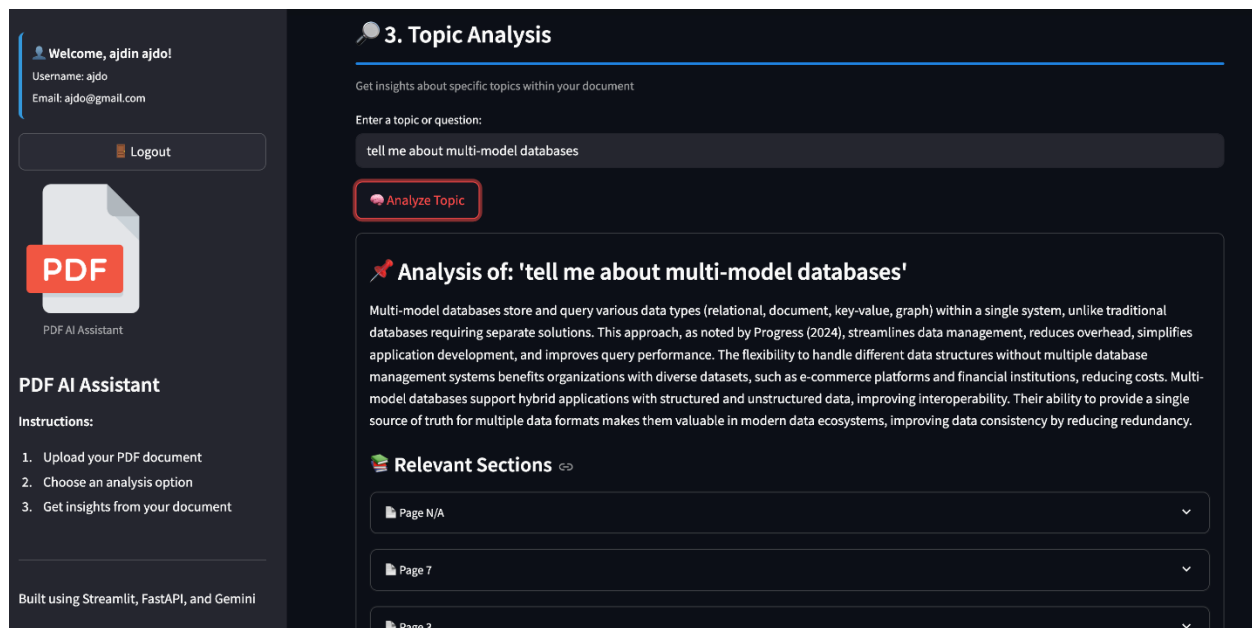


Figure 19: Topic Analysis

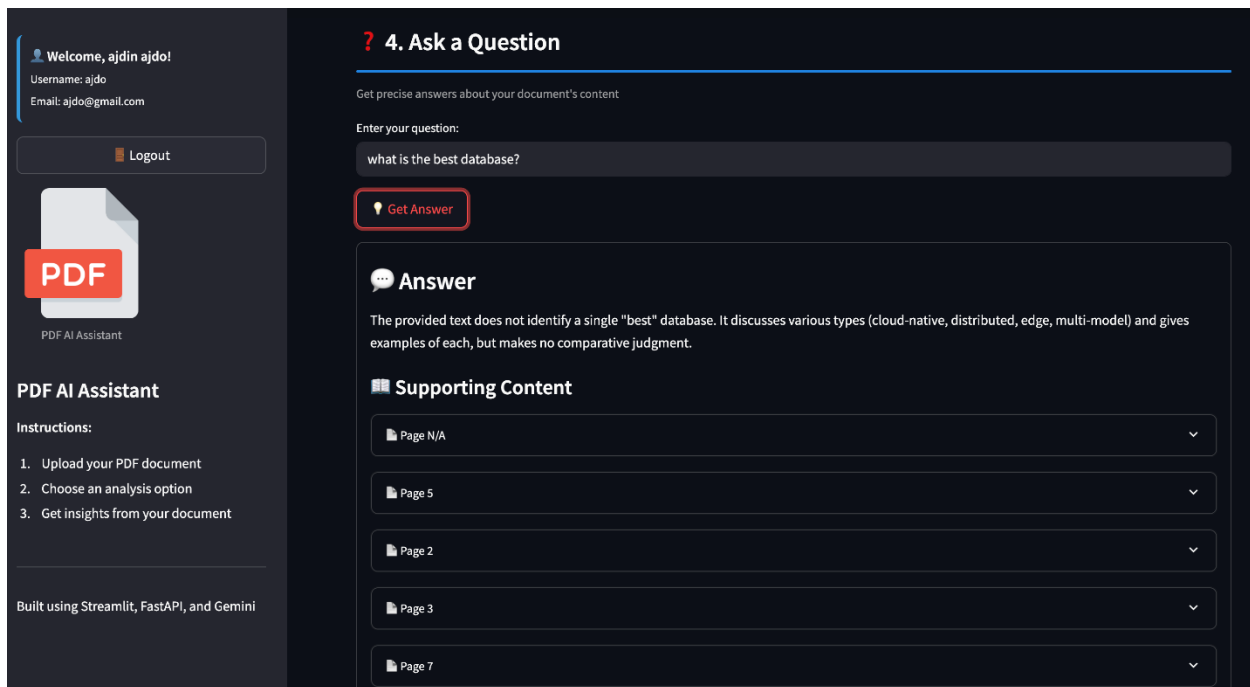


Figure 20: Asking a Question

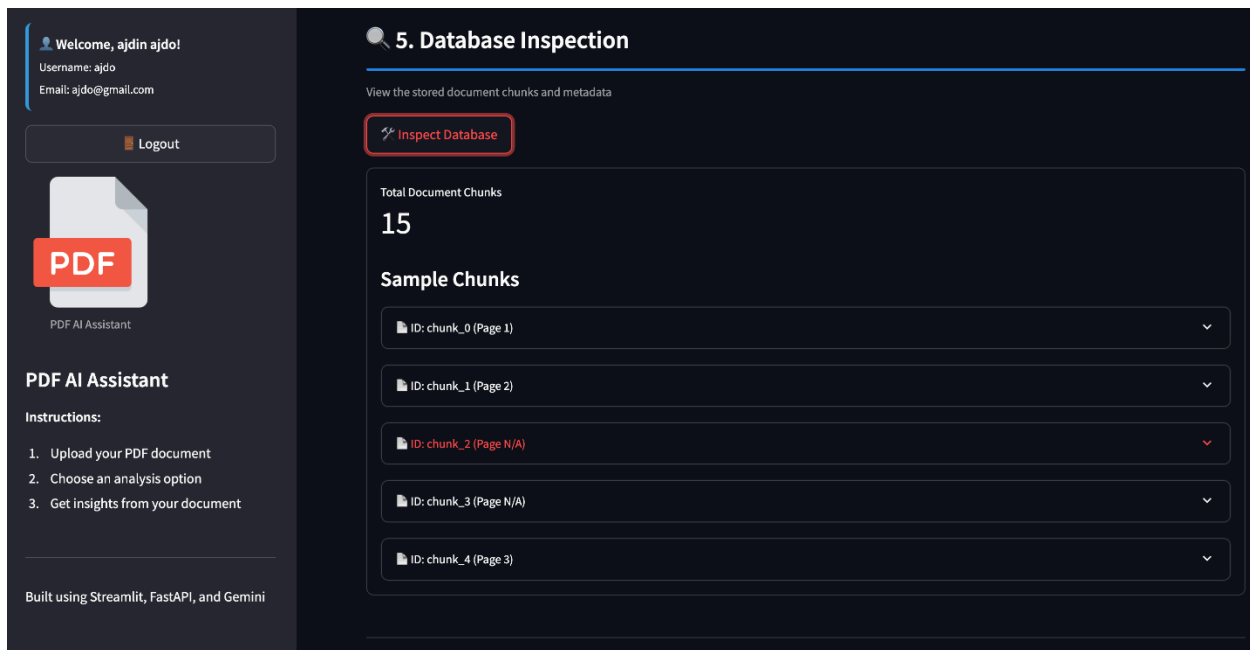


Figure 21: Inspection of the Database

6. Security and Strengths

This system's architecture placed a high emphasis on security and authentication, especially given how it manages private user data and AI-powered document interaction. Because all user passwords are kept in SHA-256 hashed format, the SQLite database is password secure. This would suggest that the non-hashed passwords will never be recovered, even in the event that the database is stolen. The system implements strict input validation procedures in addition to hashing. Regular expressions help ensure that email addresses and phone numbers adhere to prearranged, standard formats. Email addresses are checked using a general-purpose pattern to reject incorrect input, and phone numbers are compared to international dialing rules.

By placing special restrictions on the email and username fields in the SQLite database structure, additional integrity checks are implemented. By preventing the creation of multiple accounts, the restrictions lessen dispute and impersonation. Although documents and vector embeddings are handled efficiently on the ChromaDB side, there is a drawback because there is no explicit user-based separation. All document chunks are stored in a single collection.

Together, these multi-tier protections across the database, backend, and API make the system secure against common attack vectors and abuse, but still user-friendly and responsive.

The database architecture, which divides concerns in sections for better maintainability and scalability, is one of the project's key advantages. Because it makes identity storage quick, lightweight, and simple to connect with web-based user interfaces, SQLite is used to manage users and authentication. On the other hand, ChromaDB is used specifically because of its advantages in vector-based document retrieval, which allows for powerful AI question-answering and summarization.

Lastly, the application has strong layers of validation at several levels. Streamlit was used to develop the frontend, which features early validation of user input. The database layer employs restrictions and regex enforcement to prevent the saving of malicious or invalid inputs, while the FastAPI-powered backend facilitates validation at a lower level through schema models and exception handling. This multi-level validation significantly lowers the possibility of incorrect data entry, improves user experience, and adds to the platform's overall stability and security.

7. Conclusion and Results

In order to implement an AI-driven document summarization system, the project effectively demonstrates the practical application of both older and more recent database concepts: ChromaDB for vector-based unstructured document search and SQLite for structured user data. By combining these technologies with Google's Gemini AI models, the system provides intelligent question-answering capabilities based on semantic comprehension as well as accurate full-document and topic-oriented summaries.

Results demonstrate that the system can process PDFs, register users, securely log users in, and interact in real time with stored content in a flexible and connected way. Key points can be efficiently extracted from lengthy documents using the PDF summary tool, and topic-based summarization and highly relevant RAG-style question answering are made possible by the semantic search pipeline.

For further improvements, users would be able to engage with an interface to manage papers, track uploaded files, remove outdated information, or view usage history. Governance of personal data would be possible and usability would be enhanced.

Overall, this project builds a strong foundation for an AI summarizer that works well with both unstructured and structured data. With some improvements in user management, security, and scalability, it could become a reliable solution for understanding documents in real-world use.

Table of Figures

Figure 1: Schema of SQLite Database	5
Figure 2: Insight into SQLite Database	6
Figure 3: Storage Structure.....	6
Figure 4: Creating a new user in the database	8
Figure 5: Validate username method	8
Figure 6: Method that verifies the user.....	9
Figure 7: Usage of text splitter.....	9
Figure 8: Document update logic	10
Figure 9: The question-answering logic	11
Figure 10: Configuration of Gemini and Embedding Function	12
Figure 11: Summarize-pdf	13
Figure 12: Summarize-topic	13
Figure 13: Ask-pdf.....	14
Figure 14: Welcome Page of the Application.....	15
Figure 15: Registration for AI Assistant.....	15
Figure 16: Upload PDF interface while processing.....	16
Figure 17: Interface after successfully processing the PDF	16
Figure 18: Document Summary	17
Figure 19: Topic Analysis.....	17
Figure 20: Asking a Question	18
Figure 21: Inspection of the Database.....	18

References

SQLite Documentation. <https://www.sqlite.org/docs.html>

Chroma: The AI-native open-source embedding database. <https://www.trychroma.com/>

Google AI Studio – Gemini API documentation. <https://aistudio.google.com/app>

FastAPI Documentation. <https://fastapi.tiangolo.com/>

Streamlit Documentation. <https://docs.streamlit.io/>

LangChain Documentation – Text Splitting and Chunking. <https://docs.langchain.com/docs/>

OpenAI Documentation. <https://platform.openai.com/docs>